SMART CITY TRAFFIC MANAGEMENT AND ROUTE OPTIMIZATION SYSTEM

CAPSTONE PROJECT REPORT

Submitted by: Ujjwal Chauhan

Roll number: 2401201059

Course: BCA AI & DS | Semester: 5

University: K.R. Mangalam University

Faculty: Aarti Mam

Submission date: 02 November 2026

Abstract

This project presents a graph-based smart-city traffic management and route-optimization prototype. City locations are vertices and roads are weighted, undirected edges. The C++17 application implements Breadth-First Search (BFS), Depth-First Search (DFS), Dijkstra shortest path, and Prim Minimum Spanning Tree (MST). Users can update traffic-delay values and run a repeated-execution benchmark. A standalone interactive browser dashboard visualizes the same synthetic seven-location network and highlights traversal, recommended routes, and MST edges. Nine automated black-box integration tests passed. The project is an educational prototype and does not use live traffic feeds.

1. Problem Statement

Urban commuters need useful routes between locations, while city administrators need a way to reason about connectivity and the cost of connecting infrastructure. Manual route selection can ignore traffic conditions, and infrastructure planning requires a systematic method to connect locations efficiently. The project models the road network as a graph and applies standard graph algorithms to support these decisions.

2. Objectives

- Represent city locations and roads using an adjacency-list graph.
- Traverse connected locations using BFS and DFS.
- Recommend a low-cost route using Dijkstra's algorithm and road/traffic weights.
- Estimate a low-cost network connection using Prim's MST.
- Allow traffic-delay values to be changed interactively.
- Report basic runtime measurements and algorithmic complexity.

3. Scope and Assumptions

The prototype uses seven fictional locations and eleven two-way roads. Each road has a positive distance value and a non-negative traffic-delay value. For the demonstration, Dijkstra uses distance + traffic delay as a composite route score; these values are illustrative and should not be interpreted as real minutes or kilometres. Prim's MST uses distance as the infrastructure cost. The system is a command-line educational prototype and does not connect to live maps or traffic services.

4. System Design

4.1 Graph representation

The city is an undirected weighted graph G = (V, E). Each vertex represents a location. Each edge stores the destination, road distance, and traffic-delay value. An adjacency list stores each road at both endpoints.

4.2 Main modules

- Location and road data: stores location names and road weights.
- Traversal module: performs BFS using a queue and DFS using recursion.
- Route optimization module: runs Dijkstra with a min-priority queue.
- Infrastructure module: runs Prim's MST using road distance.
- Traffic update module: updates the delay of an existing road in both directions.
- Benchmark module: measures repeated BFS and Dijkstra execution time on the local machine.

4.3 Algorithm workflow

Input source/destination or selected menu action → read graph data → run the selected graph algorithm → display traversal, route, MST, or timing → optionally update traffic delay and repeat.

5. Algorithms and Complexity

Algorithm	Purpose	Time complexity	Auxiliary space
BFS	Breadth-first exploration	O(V + E)	O(V)
DFS	Depth-first exploration	O(V + E)	O(V)
Dijkstra + binary heap	Minimum composite route cost	O((V + E) log V)	O(V + E)
Prim (linear key search)	Minimum spanning tree	$O(V^2 + E)$	O(V + E)

V denotes the number of vertices and E the number of edges. The implemented Prim variant scans all vertices to select the next minimum key, so it is not the heap-optimized Prim variant.

6. Implementation

The primary algorithm implementation is C++17 in src/main.cpp. The interactive menu provides location listing, BFS, DFS, Dijkstra route selection, Prim MST, traffic updates, road listing, and repeated-run performance benchmarking. The supplementary visualization/dashboard.html file is a standalone browser dashboard that displays the sample graph and interactively demonstrates BFS, DFS, Dijkstra, Prim, and traffic-delay changes. The C++ program remains the primary assessed implementation; dashboard state is independent and is not persisted to the C++ executable. The sample network is documented in data/sample_network.csv and initialized in the C++ source.

7. Testing and Validation

The project includes an automated black-box integration test suite in tests/test_cli.py. It compiles the C++ source and checks nine cases: location listing, BFS formatting, DFS, Dijkstra, Prim MST, traffic updates, invalid route IDs, and benchmark output. The recorded run in results/automated_test_results.txt passed all eight tests. Actual CLI output logs are included under results/. Screenshots from the student's own submission computer should still be added to results/screenshots before final submission.

Test ID	Test case	Expected result
T1	List locations	Seven IDs and names shown
T2	BFS/DFS from location 0	Traversal order printed
T3	Dijkstra from 0 to 4	Route and composite cost printed
T4	Prim MST	Six edges for connected seven-node network
T5	Update an existing road delay	Updated value affects later route calculation
T6	Invalid source/destination ID	Error message shown
T7	Benchmark	BFS and Dijkstra timings displayed

8. Results and Discussion

The prototype demonstrates how graph algorithms can support different city-planning tasks. BFS/DFS show network reachability and traversal. Dijkstra selects a route according to the configured composite edge score, so changing a traffic delay can change the preferred route. Prim's MST gives a set of roads connecting all locations at minimum total distance cost under the supplied graph weights. Benchmark results are machine-dependent and should be copied from an actual run.

9. Limitations

- Road data is synthetic and small; it is not a real city map.
- No live traffic, weather, incident, or road-closure feeds are connected.
- The route score combines illustrative values and requires consistent units for real use.
- The C++ core is command-line based. A supplementary browser dashboard provides interactive schematic visualization, but it is not a geographic map and does not display live road conditions.
- Benchmarking is basic and does not replace statistically controlled performance experiments.

10. Future Enhancements

- Import road networks from OpenStreetMap or an approved GIS source.
- Replace the schematic dashboard with a geographic map and display alternate routes.
- Integrate reliable real-time traffic and incident data.
- Use estimated travel time or generalized cost with consistent units.
- Store network and traffic updates in a database.
- Add larger generated graph datasets, statistically controlled benchmark reports, and deeper unit-level testing.

11. Conclusion

The Smart City Traffic Management and Route Optimization System demonstrates the application of BFS, DFS, Dijkstra's shortest path, and Prim's MST to a simplified transportation network. Interactive traffic updates and benchmarking provide a foundation for exploring routing and infrastructure decisions. The project can be extended with real road data, live traffic inputs, automated tests, and a graphical map interface.

12. Repository and Submission Checklist

☐ Upload the complete source project and this report to the required submission location.

☐ Create a GitHub repository and add the project files, README, sample data, and test plan.

☐ Compile and run the program yourself; capture screenshots of menu, route, MST, traffic update, and benchmark output.

☐ Verify the student details and update the faculty/submission-date fields if required.

☐ Add the actual public GitHub repository URL here before ZOSIMA submission: PENDING — repository not yet supplied.

☐ Confirm the repository is accessible to faculty and include the link in the ZOSIMA submission.

Appendix A: Sample locations

- 0 — Central Station
- 1 — City Hospital
- 2 — University
- 3 — Market
- 4 — Airport
- 5 — Tech Park
- 6 — Residential Area

Appendix B: Build command

```
g++ -std=c++17 -O2 -Wall -Wextra src/main.cpp -o smart_city
```

Appendix C: Interactive Visualization

The standalone dashboard is provided at visualization/dashboard.html. It uses SVG to display the seven locations and eleven undirected roads. The user can select BFS, DFS, Dijkstra, or Prim, choose a source/destination, adjust a road traffic-delay value, and highlight the resulting route or MST edges. Edge labels show distance units / traffic-delay units.

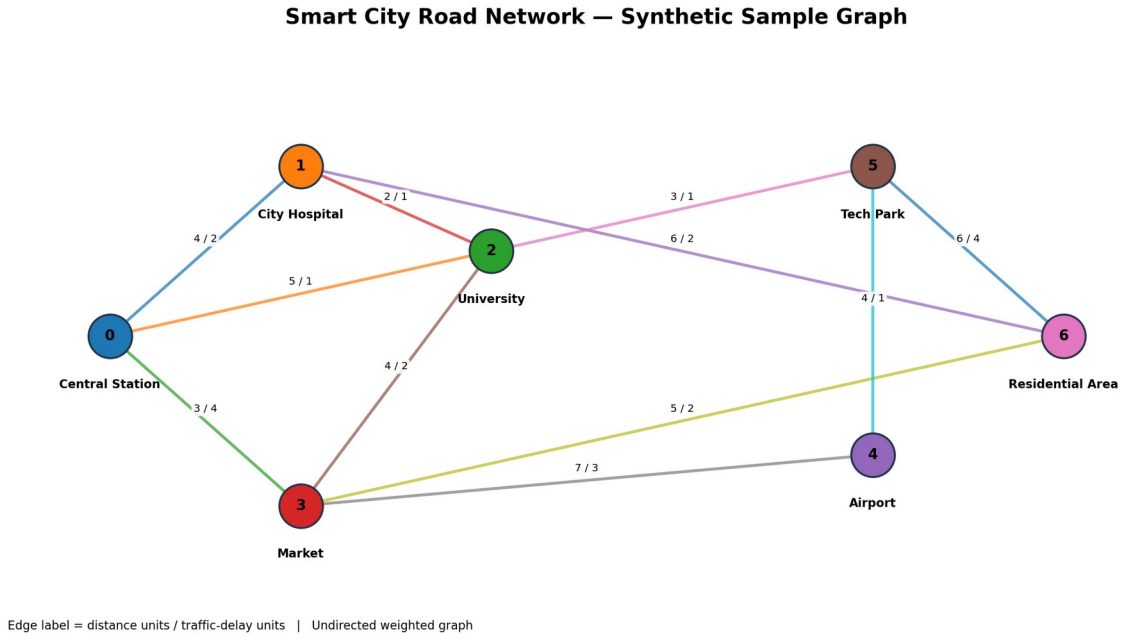


Figure C1. Schematic sample city network; edge labels are distance / traffic-delay units.

Appendix D: Requirements Traceability

Faculty requirement	Project evidence	Status
City road network represented as a graph	Adjacency-list undirected weighted graph in src/main.cpp	Implemented
Graph traversal	BFS and DFS menu operations; dashboard visualization	Implemented
Shortest path	Dijkstra with distance + traffic-delay composite score	Implemented
Minimum spanning tree	Prim MST using road distance as infrastructure cost	Implemented
Traffic condition evaluation	Runtime traffic-delay updates; dashboard slider	Implemented as simulation
Interactive visualizations	Standalone SVG browser dashboard with route/MST highlighting	Implemented; schematic, not geographic
Performance analysis	Repeated BFS and Dijkstra timings; complexity discussion	Implemented; small sample graph
Decision support	Recommended route and minimum-cost infrastructure connection	Implemented as prototype
Organized source, README, sample data	src/, visualization/, data/, tests/, results/	Included
Testing evidence	Eight automated integration tests and CLI logs	Included
Accessible GitHub repository	Actual URL must be added after repository creation	Pending student action

Complete ZOSIMA document	This report with design, implementation, tests, results and conclusion	Included

Appendix E: Automated Test Evidence

The following test suite was executed against the compiled C++17 application. All nine integration tests passed.

- test_benchmark — PASS
- test_bfs_arrow_format — PASS
- test_dfs — PASS
- test_dijkstra — PASS
- test_invalid_route_ids — PASS
- test_location_listing — PASS
- test_prim_mst — PASS
- test_traffic_update_changes_route_score — PASS
- test_traffic_update — PASS
- Summary: Ran 9 tests; OK.

The actual test-run transcript is stored in results/automated_test_results.txt. CLI demonstration outputs are stored in results/dijkstra_demo.txt, results/mst_demo.txt, results/benchmark_demo.txt, and results/traffic_update_demo.txt.

Appendix F: Demonstration Results

For source location 0 (Central Station) and destination location 4 (Airport), the sample Dijkstra run produced the route Central Station → University → Tech Park → Airport. The road-distance total was 12 units, traffic-delay total was 3 units, and combined score was 15 units. The Prim MST connected all seven locations using six edges with a total distance cost of 21 units. These values are specific to the supplied synthetic network.

The benchmark executes BFS and Dijkstra 1,000 times on the seven-vertex sample graph. One recorded run measured BFS at 0.108280 ms total and Dijkstra at 0.261517 ms total; these timings are machine- and compiler-dependent and are included only as an execution example. Refer to results/benchmark_demo.txt for the full output.

Submission items still requiring student action

- Create a GitHub repository, push the complete project folder, and replace the pending URL in this report and README with the actual accessible repository URL.
- Faculty name confirmed as Aarti Mam. Update the ZOSIMA submission date only if it differs from the report date.
- Open the dashboard and C++ application on the submission computer and capture genuine screenshots in results/screenshots/.
- Upload both the accessible GitHub link and this completed report/project file to ZOSIMA.